CONTENTS

Intermediate AI evaluation Condensed Peter Yang

How to Actually Run AI Evals: Rubrics, Golden Datasets & LLM-as-a-Judge

Walk the full evaluation loop end to end — write the rubric, hand-label a golden dataset, build an LLM judge, and prove the judge agrees with you.

For product managers and engineers shipping an LLM feature who need to know whether it is actually good enough to deploy · 27 July 2026

Every section, key point, term and question from the full lesson is here. The extended explanations and the additional examples are not.

Watch the source video Read the full lesson Read the highlights

BEFORE YOU START

- You have prompted an LLM and seen it produce a confidently wrong answer
- Comfort with spreadsheets — genuinely, that is the main tool here
- A rough idea of what a system prompt is

Overview

Every model provider tells you to evaluate your AI system, and almost nobody explains what that concretely looks like on a Tuesday afternoon. This lesson follows one worked example the whole way down: a customer-support agent for a running-shoe brand, taken from a blank prompt to a measured, trustworthy evaluation.

The core claim is unglamorous. Evaluation starts in a spreadsheet, with a human reading model outputs one at a time and arguing with a teammate about whether each one is good. Every automated technique that follows — LLM-as-a-judge, scaled scoring, continuous monitoring — is built on top of that hand-labelled foundation, and inherits its quality. Get the labels wrong and everything downstream is confidently wrong.

You will learn the four kinds of eval and when each applies, how to turn vague quality intuitions into a rubric your team can grade against, how many examples you actually need, how to encode that rubric as a judge prompt, and — the step most teams skip — how to measure whether your judge agrees with your humans before you trust a single one of its scores.

KEY TAKEAWAYS

- ✓ There are four kinds of eval: code-based assertions, human labels, LLM-as-a-judge, and user/business signals. They form a ladder of scale, and each one is validated by the one before it.
- ✓ The golden dataset is the foundation. If your hand-labelled examples are bad, every automated eval built on them is bad, no matter how sophisticated the tooling.
- ✓ A rubric turns 'is this good?' into named dimensions with written definitions — for a support agent: product knowledge, policy compliance, and tone, each graded good/average/bad.
- ✓ Start with about 10 labelled rows for internal iteration; get to 100 or more before you trust a number for production.
- ✓ More data means more confidence but slower iteration. These are orthogonal dimensions and you must consciously pick a point between them.
- ✓ An LLM judge must output an explanation, not just a score, so you can audit its reasoning rather than trusting a bare label.
- ✓ Match rate — how often the judge's label equals the human's label — is what tells you whether the judge can be trusted. A judge that scores everything 'good' has a great average and zero value.
- ✓ Disagreement between teammates during labelling is a feature: it surfaces genuine product decisions the specification never settled.

01 Why evaluation became the defining product skill

1:22

> LLMs hallucinate, so the only way to know your product works is to measure it — which is why the people selling you the models tell you to check the models' answers.

KEY POINTS

- → LLMs hallucinate as a structural property, not an occasional bug, so quality has to be measured rather than assumed.
- → Model vendors themselves push customers to build evals — the strongest available admission of how much verification is required.
- → Evals measure the quality of your AI system, which is what turns prompt changes from guesswork into engineering.
- → Without evals you cannot tell whether a change improved the system or broke something you were not watching.

The one-example trap

You ask the support agent a return-policy question, get a polished answer, and feel good about shipping. But you have sampled a single point from a distribution of thousands of possible user questions. The response you happened to see tells you almost nothing about the ones you did not.

02 The four types of eval

2:44

> Code-based, human, LLM-as-a-judge, and user evals form a ladder from cheap and narrow to expensive and real.

KEY POINTS

- → Code-based eval: binary pass/fail assertions like string presence, format checks, allowed-value checks. Cheap, deterministic, easy to generate.
- → Human eval: a PM or domain expert grading interactions by hand. The slowest per example and the source of truth for everything else.
- → LLM-as-a-judge: a model applying your rubric at scale, imitating how a human would label. The most common way to evaluate large volumes.
- → User eval: real-world signal from actual users — thumbs up/down, complaints, downstream business metrics.
- → Do not fully outsource human eval to contractors or labelling vendors: the person who owns the product judgement has to be in the spreadsheet.

A code-based eval in a few lines

The competitor-mention check needs no model and no rubric. It runs in milliseconds on every response and catches an embarrassing, unambiguous class of failure.

```
COMPETITORS = {"delta", "united", "american airlines"}

def no_competitor_mentioned(response: str, own_brand: str) -> bool:
    """Pass/fail: the reply must not steer a customer to a rival."""
    text = response.lower()
    return not any(c in text for c in COMPETITORS - {own_brand.lower()})

assert no_competitor_mentioned(
    "You can book that on our site.", own_brand="United"
)
```

03 Step one: the prompt and the context it needs

7:30

> Before you can evaluate anything you need a system to evaluate — and the prompt's quality is mostly determined by the context you feed it.

KEY POINTS

- → Identify the run-time variables first: user question, product info, policy info. These become template placeholders, not hard-coded text.
- → Prompt-generation tools give you a reasonable first draft with variables and examples already scaffolded — a fine starting point.
- → Product and policy context can be assembled quickly by having a model extract it from your own public website.
- → Automated prompt optimisation often adds bulk without adding quality; you cannot tell whether it helped without evals and real data.

The parameterised support prompt

The skeleton of the agent. Everything variable is injected per request, so the same prompt serves every customer question and can be evaluated as a fixed artefact.

```
SYSTEM_PROMPT = """You are a customer support agent for On running shoes.

Answer using only the product and policy information provided.
If the answer is not covered by the policy, say so directly
and give the customer the support contact route.

<product_info>
{product_info}
</product_info>

<policy_info>
{policy_info}
</policy_info>
"""

messages = [
    {"role": "system", "content": SYSTEM_PROMPT.format(
        product_info=catalogue, policy_info=returns_policy)},
    {"role": "user", "content": user_question},
]
```

04 Step two: write the rubric before you grade anything

15:08

> Turn 'is this good?' into named dimensions with written definitions, so that two people grading the same output reach the same label for the same reason.

KEY POINTS

- → Name dimensions, do not grade a single overall 'good'. For a support agent: product knowledge, policy compliance, tone.
- → Write an explicit definition of what good, average, and bad mean for each dimension, so labels are consistent between people.
- → Separate dimensions are diagnostic: they tell you which part of the system to fix, not just that something is wrong.
- → Draft the rubric before labelling begins — one invented mid-grading drifts and produces inconsistent labels.

A three-dimension rubric with written grade definitions

This is the whole rubric for the support agent. It is small enough to fit on one screen and specific enough that two people grading the same reply usually land on the same label.

```
Dimension: PRODUCT KNOWLEDGE
  good    – names the correct product, accurate specs and price
  average – correct product, vague or incomplete detail
  bad     – wrong product, invented specs, or claims no knowledge

Dimension: POLICY COMPLIANCE
  good    – applies the stated policy correctly, cites the rule,
            and gives the next action the policy provides
  average – correct outcome, missing the route to resolution
  bad     – misapplies the policy, invents a rule, or deflects
            to another team without giving contact details

Dimension: TONE
  good    – warm, concise, on–brand
  average – correct but formal or noticeably long
  bad     – cold, robotic, or so verbose the answer is buried
```

05 Step three: build the golden dataset by hand

16:30

> Run real questions through the system, put inputs and outputs in a spreadsheet, and grade each row against the rubric with your team.

KEY POINTS

- → One row per interaction: question, response, one grade per dimension, plus a free-text note.
- → Grading forces the team to settle product decisions the specification left open — that debate is the point, not a distraction.
- → Labelling reveals gaps in your source documents, not only in the model's behaviour.
- → Write notes on the bad rows, then have an LLM summarise them into the top few fixes.
- → A five-row dataset is enormously better than none: start somewhere and grow it.

The golden dataset as a table

Five rows of a real labelling session. The notes column is where the value accumulates — it is what you turn into next week's prompt changes.

```
| question                         | response      | prod | policy | tone | note                           |
|----------------------------------|---------------|------|--------|------|--------------------------------|
| Return Cloud Monster, 2 months?  | over 30 days  | good | good   | avg  | a bit formal                   |
| 3 weeks, lost the box            | contact us    | good | BAD    | avg  | never gives contact details;   |
|                                  |               |      |        |      | policy silent on lost box      |
| Change address, ordered 45m ago  | says past 60m | good | BAD    | good | MATH ERROR: 45 < 60            |
| Marathon, max cushioning?        | Cloud Monster | good | n/a    | avg  | very verbose                   |
| How do I track my order?         | correct link  | good | good   | good |                                |
```

06 What you find when you actually look at the data

22:44

> Real failures are specific and often boring — like a model asserting that 45 minutes is past a 60-minute window.

KEY POINTS

→ A real logged failure: the agent claimed 45 minutes was past a 60-minute window. LLMs are unreliable at arithmetic.

→ This class of bug is found by a human reading rows, not by a dashboard or a generic metric.

→ Automated evals can only catch failure modes a human first identified and encoded into a check.

→ The strongest fix is usually to move deterministic work out of the model — compute the comparison in code and pass in the result.

Removing arithmetic from the model

Rather than instructing the model to be careful with numbers, compute the eligibility in code and hand the model a fact it cannot get wrong.

```
from datetime import datetime, timedelta

def cancellation_context(order_placed_at: datetime) -> str:
    elapsed = datetime.now() - order_placed_at
    eligible = elapsed < timedelta(minutes=60)
    mins = int(elapsed.total_seconds() // 60)
    return (
        f"Order placed {mins} minutes ago. "
        f"Cancellation window is 60 minutes. "
        f"ELIGIBLE_FOR_CANCELLATION: {eligible}"
    )

# The model now reports a boolean it was handed,
# instead of performing a comparison it may get wrong.
```

07 How many examples do you actually need?

24:49

> Ten rows to start iterating internally, a hundred or more before you trust the number for

> production — and the answer is fundamentally a statistics question.

KEY POINTS

- → About 10 examples for internal testing and early iteration — enough to tell whether the approach is worth pursuing.
- → 100 or more before trusting a production decision; regulated domains need substantially more.
- → It is fundamentally a statistics question: how much data before the measurement is trustworthy?
- → Speed of iteration and confidence in the result are orthogonal — more data means more confidence and slower loops.
- → Early on you change the prompt constantly, so large manual passes on every change are impractical.

Matching dataset size to the decision

The size of the golden set should follow the weight of the decision it supports, not a fixed convention.

```
Decision being made          Rows   Why
---------------------------  -----  ------------------------------
Is this approach viable?     ~10    Fast loop, cheap to relabel
Which of 2 prompts is better?  ~50  Needs to separate real signal
Is this safe to launch?      100+   The number must be defensible
Regulated / clinical / legal  many  Confidence requirement is high
```

08 Step four: encode the rubric as an LLM judge

32:21

> The judge prompt is your rubric, rewritten as instructions to a model, returning a label

and — critically — an explanation.

KEY POINTS

- → The judge prompt is the rubric restated as model instructions, given the same context the human grader had.
- → Run one judge per dimension so each call makes a single focused judgement.
- → Always require an explanation with the label — it is the judge's equivalent of reviewer notes and makes the score auditable.
- → Once encoded, you can replay hundreds of examples across prompt variants or different models on a fixed question set.

A judge prompt for policy compliance

One dimension, the full context, a constrained label, and a required explanation. Structured output keeps the result parseable.

```
JUDGE_PROMPT = """You are grading a customer support reply for POLICY COMPLIANCE.

good    - applies the stated policy correctly, and gives the next
          action the policy provides (including contact details)
average - correct outcome, but missing the route to resolution
bad     - misapplies the policy, invents a rule, or deflects to
          another team without giving contact details

<question>{question}</question>
<policy>{policy_info}</policy>
<response>{response}</response>

Return JSON: {{"label": "good|average|bad", "explanation": "..."}}
Explain your reasoning before committing to the label.
"""
```

09 Step five: judge the judge with match rate

36:29

Compare the judge's labels against your human labels and measure how often they agree — because a judge that scores everything 'good' looks great and tells you

> nothing.

KEY POINTS

- → Match rate = the fraction of examples where the judge's label equals the human's label, computed per dimension.
- → A judge that labels everything 'good' produces a flawless-looking dashboard and zero information — only human labels expose this.
- → Per-dimension match rate localises the fault: product knowledge matched 100%, tone matched once, so the tone judge is what needs fixing.
- → Fix a disagreeing judge by making its prompt stricter on whatever the humans were penalising — here, verbosity.
- → Validate the judge on 5–10 examples first, then scale to 100 once it tracks your humans.

Computing match rate per dimension

A few lines that turn two label columns into the number that decides whether you can trust the judge.

```
def match_rate(rows, dimension):
    human = f"human_{dimension}"
    model = f"judge_{dimension}"
    scored = [r for r in rows if r.get(human) and r.get(model)]
    if not scored:
        return None
    agree = sum(1 for r in scored if r[human] == r[model])
    return agree / len(scored)

for dim in ("product", "policy", "tone"):
    print(f"{dim:>8}: {match_rate(golden_set, dim):.0%}")

# product: 100%   <- judge is aligned, trust it
# policy:   80%
# tone:     20%   <- judge is broken, rewrite this prompt
```

10 Shipping: internal, then a small slice, then real users

47:34

> Offline evals get you to a candidate; dogfooding and a small A/B slice tell you whether it

> works in the world — and user signals are noisier than they look.

KEY POINTS

- Sequence: internal dogfooding, then a 1–10% A/B slice, then broader rollout with real-world monitoring.
- Dogfooding is high-value because the people using it already know what good looks like.
- A thumbs-down conflates 'the system did badly' with 'I disliked the outcome' — the signal is real but ambiguous.
- Decide in advance how you break the tie when evals look good and the business metric drops.
- The resolution to any conflict between signals is to go back and read the actual data.

The ambiguous thumbs-down

A customer asks to return shoes after 60 days. The agent correctly explains the 30-day policy, warmly and with a link to support. The customer thumbs-downs it. Every rubric dimension scores good; the user signal is negative. Both are correct measurements of different things — treating the thumbs-down as a quality failure would push you to build an agent that misstates policy to keep people happy.

Glossary

Eval

A systematic measurement of one specific aspect of an AI system's quality, such as whether a reply followed policy.

Golden dataset

A curated set of example inputs with human-assigned quality labels, used as the reference standard for measuring the system and validating automated judges.

Rubric

The named dimensions you grade on, each with written definitions of what good, average, and bad mean, so different people label consistently.

LLM-as-a-judge

Using a language model to apply your rubric to your system's outputs at scale, producing a label and an explanation the way a human grader would.

Match rate

The fraction of examples where the LLM judge's label equals the human's label. It measures whether the judge can be trusted.

Alignment (of a judge)

The state of an LLM judge agreeing with human labels often enough that its scores can be believed without re-checking each one.

Code-based eval

A deterministic pass/fail check written in ordinary code, such as verifying a required string is present or a field holds an allowed value.

User eval

Quality signal gathered from real users in production — explicit thumbs up/down, complaints, or downstream business metrics.

Dogfooding

Using your own product internally before exposing it to customers, so the people who know what good looks like find the problems first.

Check yourself

Answer first, then open each one.

Your LLM judge reports that 100% of responses are good. Why is this a warning sign rather than a success?

Because a judge that never assigns a bad label is indistinguishable from a judge that is not working. Real systems produce a mix of quality, so uniform approval suggests the judge prompt is too lenient or too vague to discriminate. You can only detect this by comparing against human labels and computing match rate — the dashboard alone would look perfect.

Why grade three separate dimensions instead of a single overall quality score?

Because separate dimensions are diagnostic. A single 'bad' tells you something is wrong; 'policy compliance bad, product knowledge good' tells you the failure is in how rules are applied rather than in the catalogue, and those have completely different fixes. Aggregate scores also hide compensating errors, where a strong dimension masks a weak one.

Why should you start with roughly 10 labelled examples rather than going straight to 100?

Because confidence and iteration speed pull against each other. Early on you are changing the prompt constantly, and hand-labelling 100 rows on every change would make the loop unusably slow. Ten rows is enough to expose obvious failure modes and validate a judge's alignment; you scale to 100 once the system stabilises and you need a number you would defend.

A user gives a thumbs-down to a reply that your rubric scores as good on every dimension. What has actually been measured?

Two different things. The rubric measured whether the system behaved as designed — correct policy, clear communication, appropriate tone. The thumbs-down measured the user's satisfaction with the outcome, which may simply be that they disliked being told no. Treating the thumbs-down as a quality failure would train the system to misstate policy in order to please people.

Why must an LLM judge return an explanation rather than just a label?

Because a bare label is unauditable — you cannot distinguish a correct judgement from a lucky one, or diagnose why the judge disagrees with your humans. The explanation plays the role a human reviewer's notes play: when match rate is low, reading the explanations on the mismatched rows is what tells you how to rewrite the judge prompt.

Why is the golden dataset described as the foundation that everything else inherits from?

Because the LLM judge is validated against human labels, and the human labels come from the golden dataset. If those labels are inconsistent or reflect criteria nobody agreed on, the judge will be aligned to noise, and every scaled measurement built on it will be confidently wrong while appearing rigorous.

Where to go next

→ Take a feature you have already shipped, write a three-dimension rubric for it, and hand-label 10 real logged interactions before reading any dashboard.

→ Build one judge prompt for your weakest dimension, run it on those same 10 rows, and compute the match rate against your own labels.

→ Find a failure in your logs that a substring assertion would have caught, and add that assertion as a code-based eval that runs on every change.

→ Identify one place where your model performs arithmetic or date comparison, and move that computation into code that hands the model a precomputed fact.

→ Agree with your team, in writing and in advance, on how you will resolve a conflict between a good eval score and a declining business metric.
